

GPU Speed Of Light Throughput

GPU Throughput Rooflines

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

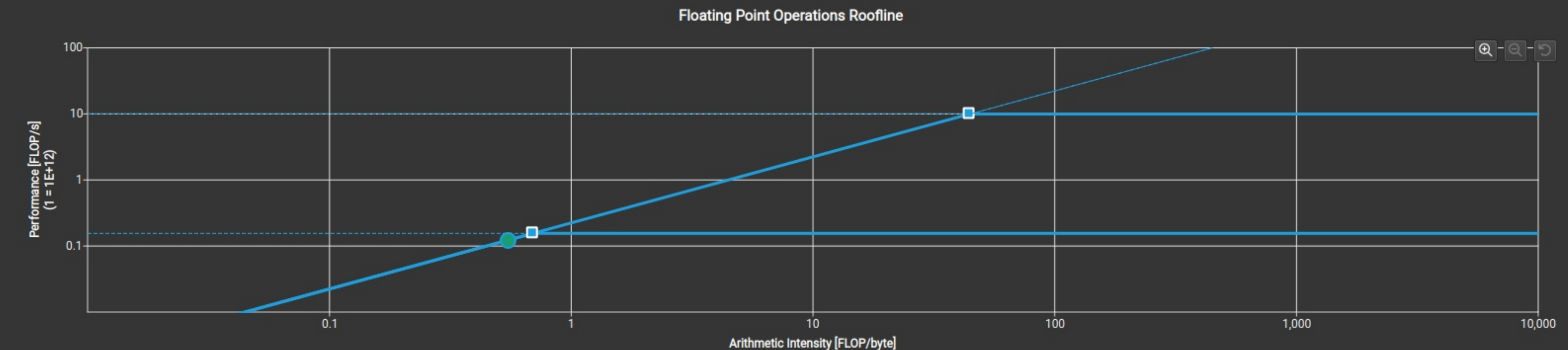
Compute (SM) Throughput [%]	24.52	Duration [usecond]	179.71
Memory Throughput [%]	94.87	Elapsed Cycles [cycle]	291,018
L1/TEX Cache Throughput [%]	24.76	SM Active Cycles [cycle]	288,058.54
L2 Cache Throughput [%]	25.43	SM Frequency [cycle/nsecond]	1.62
DRAM Throughput [%]	94.87	DRAM Frequency [cycle/nsecond]	6.99

High Throughput

The kernel is utilizing greater than 80.0% of the available compute or memory performance of the device. To further improve performance, work will likely need to be shifted from the most utilized to another unit. Start by analyzing DRAM in the [Memory Workload Analysis](#) section.

Roofline Analysis

The ratio of peak float (fp32) to double (fp64) performance on this device is 64:1. The kernel achieved 1% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the [Kernel Profiling Guide](#) for more details on roofline analysis.



PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [usecond]	4	# Pass Groups	2
Maximum Buffer Size [Mbytes]	4	Dropped Samples [sample]	0

Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.68	SM Busy [%]	17.36
Executed Ipc Active [inst/cycle]	0.69	Issue Slots Busy [%]	17.36
Issued Ipc Active [inst/cycle]	0.69		

Low Utilization

Est. Local Speedup: 89.47%

All compute pipelines are under-utilized. Either this kernel is very small or it doesn't issue enough warps per scheduler. Check the [Launch Statistics](#) and [Scheduler Statistics](#) sections for further details.

Pipe Utilization (% of active cycles)

Pipe Utilization (% of peak instructions executed)

Memory Workload Analysis

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/second]	212.26	Mem Busy [%]	18.39
L1/TEX Hit Rate [%]	11.07	Max Bandwidth [%]	94.87
L2 Hit Rate [%]	1.52	Mem Pipes Busy [%]	24.52
L2 Compression Success Rate [%]	0	L2 Compression Ratio	0

L1TEX Global Store Access Pattern

Est. Speedup: 18.39%

The memory access pattern for global stores to L1TEX might not be optimal. On average, only 8.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the [Source Counters](#) section for uncoalesced global stores.

Shared Memory					
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	384,000	384,000			
Shared Load Matrix	0	0	448,524	6.42	0
Shared Store	100,000	100,000	100,000	0.36	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	56,065	1.88	0
Total	484,000	484,000	604,589	8.66	0

L1/TEX Cache											
	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2	Returns to SM
Local Load	0	0	0	0	0	0	0	0			
Global Load	324,000	324,000			1,296,000	4		41,472,000	1,161,316	16.63	39
Global Load To Shared Store (access)	0	0	383,734	5.50	0	0	10.42	0			
Global Load To Shared Store (bypass)	0	0	0	0	0	0	-	0	0	0	
Surface Load	0	0	0	0	0	0	0	0	0	0	
Texture Load	0	0	0	0	0	0	0	0	0	0	
Global Store	16,000	16,000	16,000	0.23	16,000	1	69.99	512,000	5,877	0.08	
Local Store	0	0	0	0	0	0	0	0			
Surface Store	0	0	0	0	0	0	0	0	0	0	
Global Reduction	0	0	0	0	0	0	0	0	0	0	
DSMEM Reduction						-	-		0	0	
Surface Reduction	0	0	0	0	0	0	0	0	0	0	
Global Atomic ALU	0	0	0	0	0	0	0	0	0	0	
Global Atomic CAS	0	0	0	0	0	0	0	0	0	0	see i
Surface Atomic ALU	0	0	0	0	0	0	0	0	0	0	
Surface Atomic CAS	0	0	0	0	0	0	0	0	0	0	see i
Loads	324,000	324,000	383,734	5.50	1,296,000	4	10.42	41,472,000	1,161,316	16.63	39
Stores	16,000	16,000	16,000	0.23	16,000	1	69.99	512,000	5,877	0.08	
Atomics & Reductions	0	0	0	0	0	0	0	0	0	0	

L2 Cache											
	Requests	Sectors	Sectors/Req	% Peak	Hit Rate	Bytes	Throughput	Sector Misses to Device	Sector Misses to System	Sector Misses to Peer	
L1/TEX Load	435,267	1,161,812	2.67	12.82	0.81	37,177,984	206,875,356,125.36	1,152,036	0	0	
L1/TEX Store	5,796	5,819	1.00	0.13	100	186,208	1,036,146,723.65	0	0	0	
L1/TEX Atomic ALU	0	0	0	0	0	0	0	0	0	0	
L1/TEX Atomic CAS	0	0	0	0	0	0	0	0	0	0	
L1/TEX Reduction	0	0	0	0	0	0	0	0	0	0	
L1/TEX Total	439,091	1,166,364	2.66	12.87	1.31	37,323,648	207,685,897,435.90	1,152,036	0	0	
ECC Total	-	0	-	0	-	0	0	0	-	-	
GPU Total	439,564	1,171,011	2.66	12.92	1.52	37,472,352	208,513,354,700.85	1,152,054	1	0	

L2 Cache Eviction Policies							
	First	Hit Rate	Last	Hit Rate	Normal	Hit Rate	Hit Rate
L1/TEX Load	0	0	0	0	1,160,632	0.74	0
L1/TEX Store	0	0	0	0	5,934	100	0
L1/TEX Atomic	0	0	0	0	0	0	-
L1/TEX Total	0	0	0	0	1,167,338	1.31	0
GPU Total	3,114	85.00	0	0	1,167,929	1.36	0

Device Memory			
	Sectors	% Peak	Bytes
Load	1,152,228	91.71	36,871,296
Store	39,808	3.17	1,273,856
Total	1,192,036	94.87	38,145,152

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	9.71	No Eligible [%]	79.52
Eligible Warps Per Scheduler [warp]	0.29	One or More Eligible [%]	20.48
Issued Warp Per Scheduler	0.20		

Issue Slot Utilization

Est. Local Speedup: 5.13%

Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 4.9 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this kernel allocates an average of 9.71 active warps per scheduler, but only an average of 0.29 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the [Warp State Statistics](#) and [Source Counters](#) sections can help, too.

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	47.43	Avg. Active Threads Per Warp	28.07
Warp Cycles Per Executed Instruction [cycle]	47.74	Avg. Not Predicated Off Threads Per Warp	26.69

Long Scoreboard Stalls

Est. Speedup: 5.13%

On average, each warp of this kernel spends 38.1 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 80.4% of the total average of 47.4 cycles between issuing two instructions.

Warp Stall

Check the [Warp Stall Sampling \(All Samples\)](#) table for the top stall locations in your source based on sampling data. The [Kernel Profiling Guide](#) provides more details on each stall reason.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcodes' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	4,768,000	Avg. Executed Instructions Per Scheduler [inst]	49,666.67
Issued Instructions [inst]	4,799,745	Avg. Issued Instructions Per Scheduler [inst]	49,997.34

FP32 Non-Fused Instructions

Est. Speedup: 1.36%

This kernel executes 304000 fused and 308000 non-fused FP32 instructions. By converting pairs of non-fused instructions to their [fused](#), higher-throughput equivalent, the achieved FP32 performance could be increased by up to 25% (relative to its current performance). Check the Source page to identify where this kernel executes FP32 instructions.

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	4,000	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	47	Static Shared Memory Per Block [kbyte/block]	1.54
Block Size	128	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	512,000	Driver Shared Memory Per Block [kbyte/block]	1.02
Waves Per SM	16.67	Shared Memory Configuration Size [kbyte]	65.54
Uses Green Context	0	# SMs [SM]	24

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	83.33	Block Limit Registers [block]	10
Theoretical Active Warps per SM [warp]	40	Block Limit Shared Mem [block]	25
Achieved Occupancy [%]	68.85	Block Limit Warps [block]	12
Achieved Active Warps Per SM [warp]	33.05	Block Limit SM [block]	24

Achieved Occupancy

Est. Speedup: 5.13%

The difference between calculated theoretical (83.3%) and measured achieved occupancy (68.9%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the [CUDA Best Practices Guide](#) for more details on optimizing occupancy.

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	288,058.54	Average L1 Active Cycles [cycle]	288,058.54
Average L2 Active Cycles [cycle]	237,467.06	Average SMSP Active Cycles [cycle]	244,146.97
Average DRAM Active Cycles [cycle]	1,192,036	Total SM Elapsed Cycles [cycle]	6,981,832
Total L1 Elapsed Cycles [cycle]	6,981,832	Total L2 Elapsed Cycles [cycle]	4,532,528
Total SMSP Elapsed Cycles [cycle]	27,927,328	Total DRAM Elapsed Cycles [cycle]	5,025,792

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	628,000	Branch Efficiency [%]	95.51
Branch Instructions Ratio [%]	0.13	Avg. Divergent Branches	166.67

Uncoalesced Shared Accesses

Est. Speedup: 11.56%

This kernel has uncoalesced shared accesses resulting in a total of 64000 excessive wavefronts (12% of the total 548000 wavefronts). Check the L1 Wavefronts Shared Excessive table for the primary source locations. The [CUDA Best Practices Guide](#) has an example on optimizing shared memory accesses.

L1 Wavefronts Shared Excessive

Location	Value	Value (%)
0x7af53bbe9bf0 in gemv2T_kernel_val	16,000	25
0x7af53bbe9b90 in gemv2T_kernel_val	16,000	25
0x7af53bbe9b80 in gemv2T_kernel_val	16,000	25
0x7af53bbe9b70 in gemv2T_kernel_val	16,000	25
0x7af53bbe9d90 in gemv2T_kernel_val	0	0

Follow the [rules outputs](#) to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel. You could also disable [individual sections](#) to focus on selected performance aspects and make profiling faster.